

```
+++ date = '2026-01-01T12:44:47+10:00' draft = false title = 'Model Context Protocol' tags = ['MCP', 'Agents', 'Design Patterns', 'AI'] summary = "Model Context Protocol - Notes, Best Practices, Design Patterns." +++
```

Introduction

Model Context Protocol is an open standard protocol that aims to provide a universal approach for applications to provide context to language models. One advantage of this allows different clients to consume servers built by different vendors that too without needing to worry about compatibility issues.

MCP is built around Javascript Object Notation Remote Procedure Call (JSON-RPC 2.0) and so it's transport agnostic. It just defines the schema driven messages for client server communication.

Transports

It is usually implemented as:

- Streamable HTTP
- STDIO
- Server Sent Events (SSE) + HTTP.
- WebSockets (experimental - as persistent real time connectivity may be an overkill) etc.

All the above are common in that:

- they can operate on streams and support bidirectional communication.
- can handle reconnection handling.
- they can accomodate JSON-RPC messages in their payloads.
- Session Management
- supported in mcp.json file (websockets - experimental).
- Supergateway which is an open source CLI which acts as bridge or adaptor convertor for MCP servers (e.g. STDIO->Streamable HTTP, STDIO->WS). It allows running with Docker.

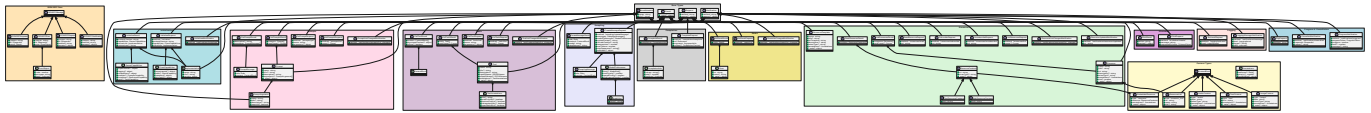
There might be some Transports not ideal for MCP:

- gRPC as it may require extra work to directly support JSON-RPC
- Messaging Queues (without reply queues) as they dont support RPC pattern
- UDP (without custom bidirectional protocol)
- ICMP - No Session Management
- FTP - Block Transfer Protocol
- Email - Async, unidirectional and slow.
- Server Sent Events (without HTTP) - Unidirectional and may not support all features of JSON-RPC.
- Webhooks
- Polling based HTTP etc.

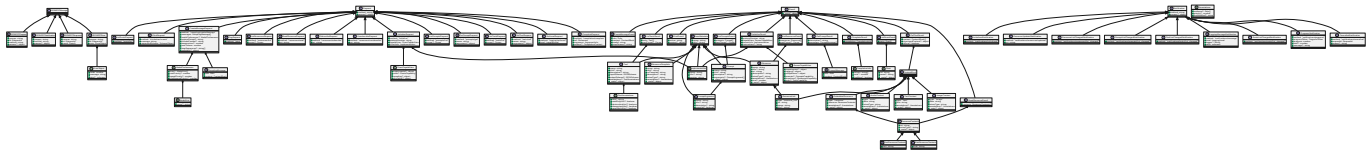
[MCP Implementation and list of Servers](#)

Communication Message Spec

Class Diagram Grouped



Class Diagram Original



Streamable HTTP

This protocol has replaced the Server Sent Events (SSE) + HTTP transport as the recommended transport for MCP because:

- **Single Endpoint Simplicity:** client and servers communicate via single endpoint (e.g., /mcp) for all interactions, eliminating the need for separate endpoints for requests and notifications.
- **Resumability:** Support for resumable sessions using headers like Last-Event-ID, Mcp-Session-ID allowing clients to recover from disconnections without losing context.
- **Compatibility:** support for Modern HTTP Infrastructure - loadbalancers, proxies, API Gateways, CDNs etc where SSEs may face challenges.
- **Bidirectional Communication:** SSE is unidirectional, Streamable HTTP can be upgraded to support bidirectional communication, allowing servers to send messages to clients without the need for clients to poll for updates. This makes it apt for agent-to-agent or client-server interaction.
- **Future Proofing:** It is modular and extendable and designed for stateless or session based models

Accept Header:

- **Accept: application/json:** Tells Server that client can handle batch JSON Response. The entire response is buffered and sent as a complete JSON object once it's fully ready. The client waits for the whole thing before processing.
- **Accept: text/event-stream:** Tells Server that client can handle Streamed events (via SSE). Uses Server-Sent Events (SSE) to stream data incrementally. As soon as each chunk/event is available, it's sent to the client immediately, allowing for real-time progressive updates

Resumability:

- If client loses connection with server while data is transfered, reconnects with server to resume the exchange from where it left off.
- This is achieved using headers like Last-Event-ID, Mcp-Session-ID etc.
- In the context of MCP its only supported for Streamable HTTP (though possible with both SSE & Streamable HTTP)

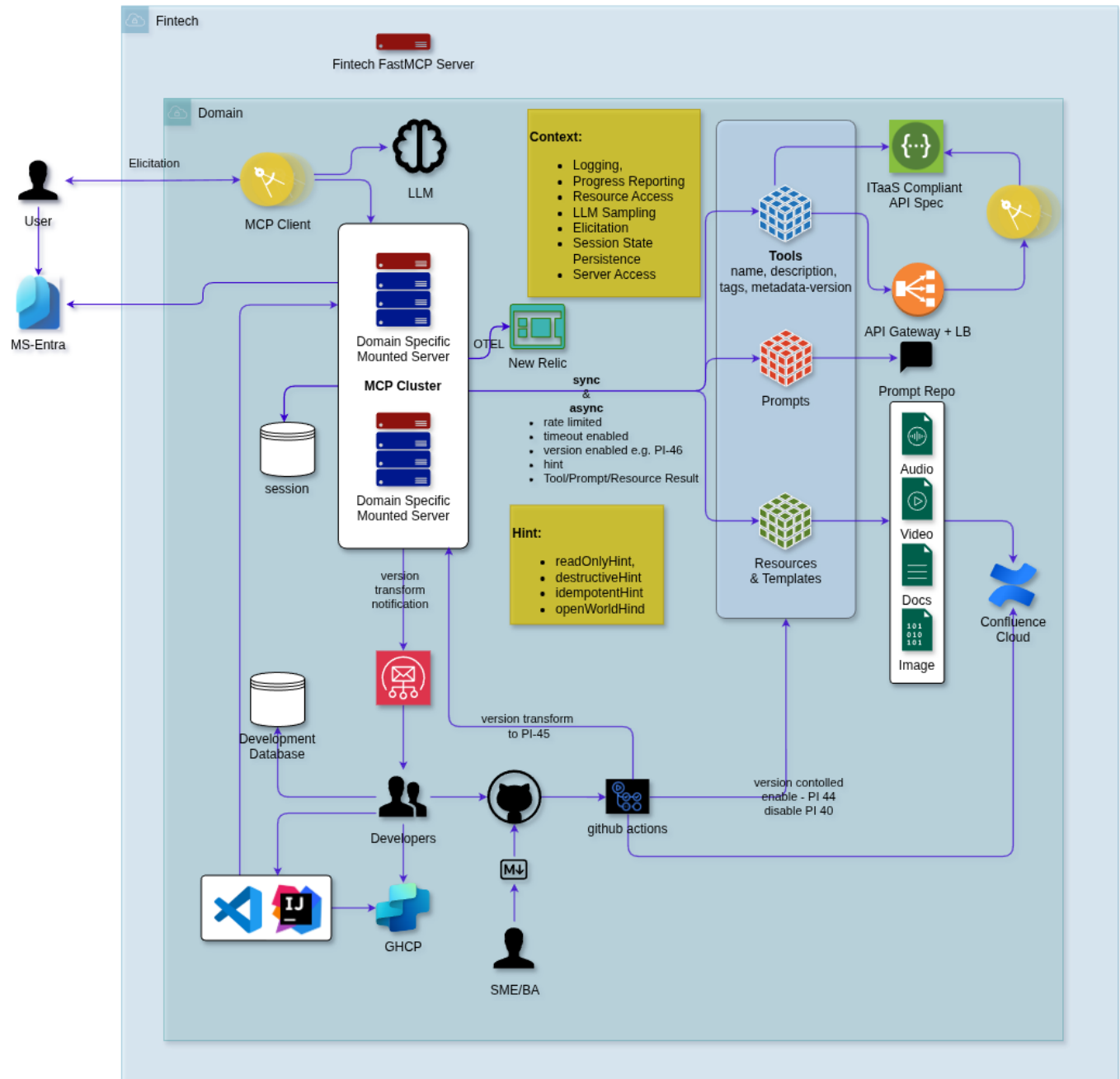
Challenges of MCP:

- Give it too much tools, verbose responses, and it get overwhelmed and lead to slower response times. Give it too little and it can't do the job.
- Protocol is complex and have layers of serialization, validation and error handling.
- MCP and libraries are fast evolving and changing.

- e.g. SSE transport was introduced somewhere in Nov 2024 and deprecated in favor of Streamable HTTP in March 2025.
- [Upgrade Guide](#)

MCP Usecase

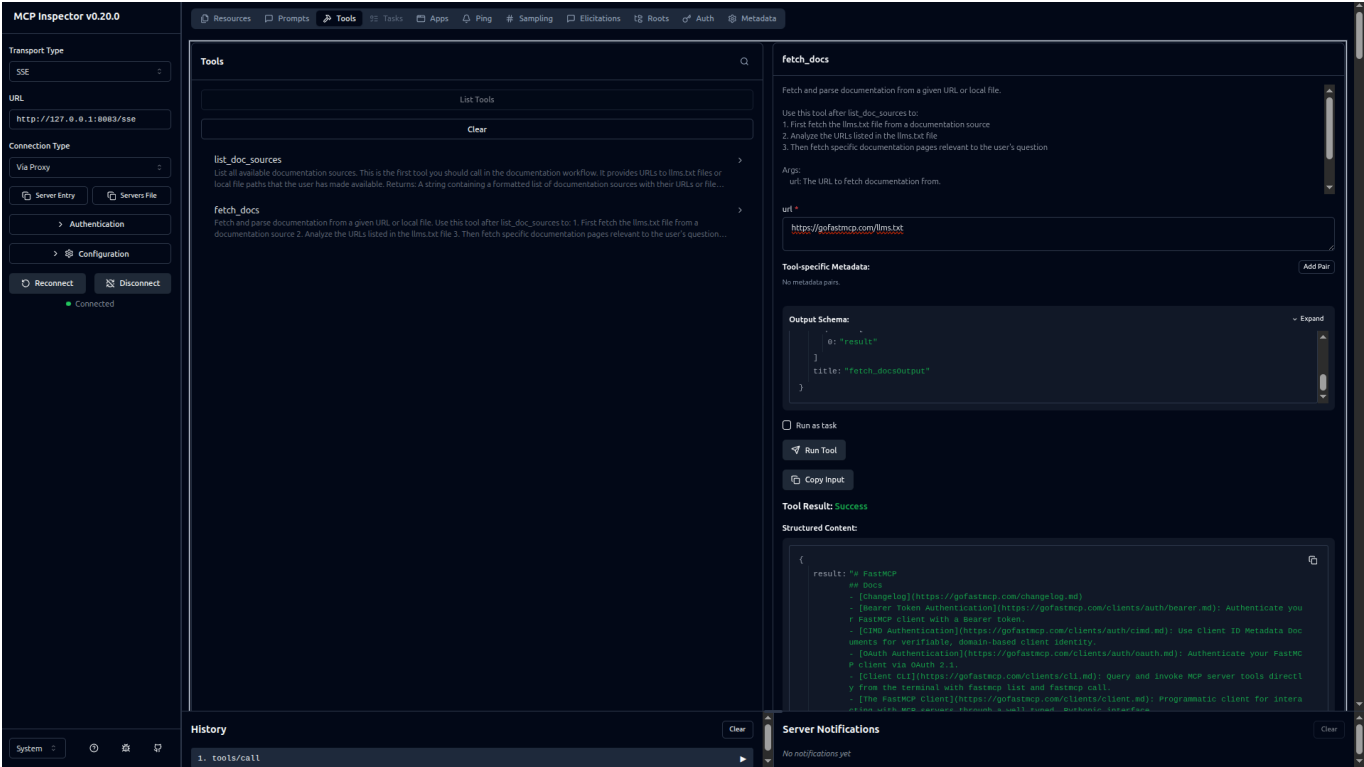
Architecture Diagram



Exposing LLM.txt as MCP Server



Testing MCP Server with LLM.txt Inspector



Testing MCP Server with PyCharm MCP Plugin

__init__.py

README.md

mcp.json

.gitignore

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

```
{
  "servers": {
    "fast-mcp": {
      "command": "/home/ai/.local/bin/uvx",
      "args": [
        "--from",
        "mcpdoc",
        "mcpdoc",
        "--urls",
        "FastMCP:https://gofastmcp.com/llms.txt",
        "--transport",
        "stdio"
      ]
    }
  }
}
```

GitHub Copilot MCP Log

Clear Logs

Filter:

Server: fast-mcp

Edit Config

[2026-02-10 12:54:57.556][info]

[fast-mcp]

Starting server fast-mcp

[2026-02-10 12:54:58.727][warning]

[fast-mcp]

[server stderr] [02/10/26 12:54:58]

INFO

Processing request of type

server.py:720

ListToolsRequest

[2026-02-10 12:54:58.730][warning]

[fast-mcp]

[server stderr]

INFO

Processing request of type

server.py:720

ListResourcesRequest

[2026-02-10 12:54:58.733][warning]

[fast-mcp]

[server stderr]

INFO

Processing request of type

server.py:720

ListResourceTemplatesRequest

[2026-02-10 12:54:58.777][info]

[fast-mcp]

Discovered 0 resources

[2026-02-10 12:54:58.777][info]

[fast-mcp]

Discovered 0 resource templates

[2026-02-10 12:54:58.777][info]

[fast-mcp]

Connection state: Running

[2026-02-10 12:54:58.777][warning]

[fast-mcp]

[server stderr]

INFO

Processing request of type

server.py:720

ListPromptsRequest

References

2.  [Learn model context protocol with python](#)
 - [Source Code](#)
3. [Official Documentation](#)